

09. 텍스트 처리 응용

NLP 파이프라인 전체를 훑고, 규칙 기반 감성 분석에서 LSTM, 기계 번역(seq2seq)까지 이어진다.

1. NLP 파이프라인 5단계

텍스트가 모델에 들어가기까지의 흐름이다.

```
raw_text    = "이 영화는 정말 재미있어요. 배우들 연기가 좋습니다."
tokens      = ["이", "영화는", "정말", "재미있어요", ".", "배우들", "연기가", "좋습니다", "."]
cleaned     = ["영화", "정말", "재미있어요", "배우들", "연기가", "좋습니다"]
vectorized  = [45, 123, 892, 156, 234, 567]
prediction  = {"감정": "긍정", "확률": 0.94}
```

단계	하는 일	주의점
원문	수집	인코딩, 특수문자
토큰화	단어 단위로 쪼갬	한국어는 조사 처리가 관건
정제	불용어 제거, 표제어 추출	과하게 지우면 의미 손실
벡터화	숫자로 변환	사전에 없는 단어(<unk>) 처리
예측	모델 통과	확률과 라벨을 함께

한국어는 영화는 → 영화 + 는 처럼 조사가 붙어 영어보다 토큰화가 까다롭다. 실무에서는 KoNLPy, Mecab 같은 형태소 분석기를 쓴다.

2. 규칙 기반 감성 분석

딥러닝 이전에 쓰이던 방식이지만, 지금도 빠른 프로토타입이나 근거가 명확해야 하는 업무에서 쓴다.

```
class BusinessSentimentAnalyzer:
    def __init__(self):
        self.positive_keywords = ["좋", "훌륭", "대박", "강추", "꿀템", "만족"]
        self.negative_keywords = ["나쁘", "별로", "짜증", "실망", "최악", "환불"]

    def analyze_sentiment(self, text):
        positive_count = sum(1 for w in self.positive_keywords if w in text)
        negative_count = sum(1 for w in self.negative_keywords if w in text)
        ...
```

키워드를 좋 처럼 어간만 둔 게 요령이다. 좋아요, 좋네요, 좋습니다 를 한 번에 잡는다.

신뢰도 계산

```
confidence = min(0.9, 0.6 + positive_count * 0.1)
```

매칭 개수가 많을수록 확신을 높이되, min 으로 상한을 둔다. 규칙 기반은 절대 100% 확신을 주면 안 된다는 원칙이 들어간 부분이다.

예측을 행동으로 연결

이 예제의 진짜 요점은 감성 판정이 아니라 그 다음에 무엇을 할지다.

```
def _get_negative_action(self, text):
    if any(w in text for w in ["배송", "늦어요"]):
        return "물류팀 우선 처리 (배송 개선)"
    elif any(w in text for w in ["품질", "별로"]):
        return "품질 관리팀 검토 필요"
```

우선순위	조건
긴급	부정 + 신뢰도 > 0.8
높음	부정
활용	긍정 + 신뢰도 > 0.8
모니터링	그 외

"부정입니다"로 끝나지 않고 **어느 팀이 언제 처리할지**까지 내려간다. 분석 결과가 의사결정으로 이어지는 구조다.

집계

```
from collections import Counter

sentiment_stats = Counter([r["sentiment"] for r in results])
department_actions = Counter([r["action"] for r in results])

for action, count in department_actions.items():
    print(action, count)
```

`Counter` 는 리스트의 항목별 개수를 세는 도구다. 부서별 처리 건수를 집계하는 데 바로 쓰인다.

규칙 기반의 한계

한계	예시
부정 표현	" 좋지 않 아요 " → 긍정으로 오판
반어	" 참 잘하는 짓이다 "
문맥	" 비싸지만 좋다 " vs " 좋지만 비싸다 "
신조어	사전에 없으면 못 잡음

이 한계가 딥러닝 감성 분석으로 넘어가는 이유다.

3. LSTM 게이트

기본 RNN은 시점이 길어지면 앞쪽 정보가 사라진다. LSTM은 세 개의 게이트로 기억을 조절한다.

게이트	결정하는 것	비유
forget	이전 기억 중 버릴 것	지우개
input	새 정보 중 저장할 것	필기
output	기억 중 지금 꺼낼 것	발표

세 게이트 모두 sigmoid를 통과해 0~1 값을 낸다. 0이면 완전 차단, 1이면 완전 통과다. 이 비율이 학습으로 정해진다.

셀 상태(`cell`)가 장기 기억을 담고, 은닉 상태(`hidden`)가 현재 출력을 담는다. 그래서 LSTM의 반환값이 (`hidden`, `cell`) 두 개다.

4. LSTM 감성 분석기

```

class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim,
                  output_dim, n_layers = 1, dropout = 0.5):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx = 0)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim,
                             num_layers = n_layers,
                             batch_first = True, dropout = dropout)

        self.fc = nn.Linear(hidden_dim, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        embedded = self.dropout(self.embedding(x))
        lstm_out, (hidden, cell) = self.lstm(embedded)
        final_feature = hidden[-1]
        output = self.fc(final_feature)
        return output

```

짚어야 할 세 곳

padding_idx = 0 — 0번 토큰의 임베딩을 0 벡터로 고정하고 학습에서도 제외한다. 패딩이 의미를 갖지 않게 만드는 장치다.

hidden[-1] — 마지막 층의 은닉 상태다. 시점이 아니다. `hidden`의 shape가 (층수, 배치, hidden_dim) 이라 `-1`이 층 인덱스가 된다.

dropout 과 **층수** — 파이토치 RNN 계열의 `dropout`은 층과 층 사이에만 걸린다. `n_layers = 1`이면 적용될 자리가 없어 경고가 뜬다.

UserWarning: dropout option adds dropout after all but last recurrent layer, but num_layers greater than 1 is expected

가변 길이 패딩

```

from torch.nn.utils.rnn import pad_sequence

x_train = pad_sequence(x_train_seq, batch_first = True, padding_value = 0)

```

가장 긴 시퀀스에 맞춰 0으로 채운다. 길이 6, 4, 4, 5짜리 네 개를 넣으면 (4, 6)이 된다.

5. 학습과 평가

```

for epoch in range(num_epochs):
    model_lstm.train()
    total_loss = 0.0
    for batch_x, batch_y in [(x_train, y_train)]:
        optimizer.zero_grad()
        outputs = model_lstm(batch_x)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(total_loss / 1)

model_lstm.eval()
with torch.no_grad():
    outputs = model_lstm(x_train)
    _, predicted = torch.max(outputs, 1)
    accuracy = 100 * (predicted == y_train).sum().item() / y_train.size(0)

```

손실이 0.589 → 0.387로 줄고 정확도 100%가 나온다.

이 100%는 성능이 아니다. 평가에 `x_train`을 그대로 썼다. 훈련에 쓴 데이터를 다시 맞추는 건 당연하다. 샘플이 4개뿐이라 모델이 통째로 외웠을 뿐이다. 실제 성능은 **본 적 없는 데이터**로 재야 한다. 데이터가 적으면 교차검증을 쓴다.

[(x_train, y_train)] 처럼 리스트로 감싼 건 배치가 하나뿐이라 DataLoader 없이 루프 형태만 맞춘 것이다. 데이터가 커지면 DataLoader 로 바꿔야 한다.

평가 시 model.eval() + torch.no_grad() 는 항상 세트다.

6. 학습 곡선 시각화

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(epochs, train_loss, marker='o', label='Train Loss')
plt.title('훈련 손실 변화')

plt.subplot(1, 2, 2)
plt.plot(epochs, train_accuracy, marker='o', color='green')
plt.title('훈련 정확도 변화')
```

subplot(1, 2, 1) 은 "1행 2열 중 1번째"라는 뜻이다. 손실과 정확도를 나란히 보면 학습 상태를 판단하기 쉽다.

한글 폰트 설정

```
from matplotlib import rcParams
rcParams['font.family'] = 'Malgun Gothic'
rcParams['axes.unicode_minus'] = False
```

matplotlib 기본 폰트(DejaVu Sans)에는 **한글 글리프가 없다**. 설정하지 않으면 제목이 네모(□□□)로 나오고 이런 경고가 쌓인다.

```
UserWarning: Glyph 54984 (\N{HANGUL SYLLABLE HUN}) missing from font(s) DejaVu Sans.
```

HUN 은 "훈", RYEON 은 "련"이다. 어느 글자를 못 그리는지 알려준다.

axes.unicode_minus = False 를 빼면 음수 축 라벨의 빼기 기호가 깨진다. **두 줄은 세트다**.

커널을 재시작하거나 plt.style.use(...) 를 다시 호출하면 설정이 초기화되므로, 노트북 맨 위 import 셀에 두는 편이 안전하다.

읽는 법

패턴	해석
손실 계속 감소, 정확도 상승	정상 학습 중
손실이 평평	학습률이 너무 작거나 수렴
손실이 NaN	학습률이 너무 큼
train은 좋은데 val이 나빠짐	과적합 시작

7. 기계 번역 seq2seq

인코더가 원문을 벡터로 압축하고, 디코더가 그 벡터에서 번역문을 한 단어씩 만든다.

인코더

```
class Encoder(nn.Module):
    def forward(self, src):
        embedded = self.embedding(src)
        outputs, (hidden, cell) = self.lstm(embedded)
        return outputs, (hidden, cell)
```

디코더에 넘기는 건 (hidden, cell) 이다. 문장 전체가 이 상태 벡터에 압축된다.

디코더

```
class Decoder(nn.Module):
    def forward(self, trg, hidden, cell):
        trg = trg.unsqueeze(1) # (B,) → (B, 1)
        embedded = self.embedding(trg)
        output, (hidden, cell) = self.lstm(embedded, (hidden, cell))
        prediction = self.fc_out(output.squeeze(1))
        return prediction, hidden, cell
```

한 시점씩 처리한다. `unsqueeze(1)` 로 시퀀스 길이 1을 만들어 넣고, 나온 상태를 다음 시점으로 넘긴다.

교사 강요 (Teacher Forcing)

```
for t in range(1, trg_len):
    output, hidden, cell = self.decoder(input, hidden, cell)
    outputs[:, t, :] = output
    teacher_force = torch.rand(1).item() < teacher_forcing_ratio
    top1 = output.argmax(1)
    input = trg[:, t] if teacher_force else top1
```

다음 입력으로 **정답을 줄지 모델의 예측을 줄지**를 확률로 정한다.

방식	다음 입력	장단점
교사 강요	정답 <code>trg[:, t]</code>	학습이 빠르고 안정적
자유 실행	예측 <code>top1</code>	실제 추론과 같은 조건

교사 강요만 쓰면 학습 때는 잘하는데 추론에서 무너진다. 한 단어를 틀리면 그 뒤가 연쇄적으로 어긋나는데, 그 상황을 겪어본 적이 없어서다. 그래서 확률로 섞는다. 보통 0.5에서 시작해 점차 낮춘다.

손실 계산

```
output_dim = output_seq.shape[-1]
output_seq = output_seq[:, 1:, :].reshape(-1, output_dim)
trg_seq = trg_example[:, 1:].reshape(-1)
loss_seq = criterion_seq2seq(output_seq, trg_seq)
```

첫 토큰을 제외하는 이유 — 0번은 `<sos>` (시작 토큰)라 예측 대상이 아니다. 디코더 루프도 `range(1, trg_len)` 으로 1부터 시작하므로 `outputs[:, 0, :]` 는 0으로 남아 있다.

항목	shape (배치 32, 길이 12, 어휘 5000)
모델 출력	<code>(32, 12, 5000)</code>
첫 토큰 제외 후 평탄화	<code>(352, 5000)</code>
<code>trg_seq</code>	<code>(352,)</code>

$352 = 32 \times 11$ 이다. 두 값의 행 수가 맞아야 `CrossEntropyLoss` 가 동작한다.

```
criterion = nn.CrossEntropyLoss(ignore_index = 0)
```

`ignore_index = 0` 은 패딩 토큰을 손실에서 제외한다. 실제 데이터에서 필수다.

학습 전 손실은 $\ln(\text{어휘 크기})$ 근처에서 시작한다. 어휘가 5000이면 $\ln(5000) \approx 8.5$ 다. 이 값과 크게 다르면 초기화나 구현을 의심한다.

검수에서 찾은 동작 오류

1. seq2seq가 마지막 시점만 반환

```
return output # 잘못 — 루프 변수, 마지막 한 스텝
return outputs # 정상 — 전체 시퀀스
```

변수	내용	shape
<code>output</code>	마지막 시점 1개	<code>(B, output_dim)</code>
<code>outputs</code>	전체 시퀀스	<code>(B, trg_len, output_dim)</code>

`outputs` 를 루프에서 열심히 채워놓고 버리는 셈이다. 이름이 한 글자 차이라 놓치기 쉽고, 손실 계산에서 시퀀스 전체를 못 써 학습이 성립하지 않는다.

2. 슬라이싱 콜론 누락

```
output_seq[:, 1, :]      # 잘못 — 시점 1 하나만 → (32, 5000)
output_seq[:, 1:, :]     # 정상 — 1번부터 끝까지 → (32, 11, 5000)
```

trg_example 쪽은 더 직접적으로 터진다.

```
trg_example[:, 1, :]      # (32, 12) 2차원에 인덱스 3개
```

```
IndexError: too many indices for tensor of dimension 2
```

3. 평가에 훈련 데이터 사용

```
outputs = model_lstm(x_train)      # 훈련 데이터로 평가
accuracy = 100.0
```

정확도 100%가 나오지만 의미가 없다. 샘플 4개를 외운 결과다. 별도의 검증셋이 필요하다.

4. 한글 폰트 미설정

그래프 제목이 네모로 나오고 경고가 쌓인다. 커널 재시작 후 설정이 풀리는 경우가 많으니 import 셀에 고정으로 둔다.

정리 포인트

- NLP는 원문 → 토큰화 → 정제 → 벡터화 → 예측. 한국어는 조사 처리가 관건
- 규칙 기반은 어간만 키워드로 두면 활용형을 함께 잡는다
- 분석은 판정에서 끝내지 말고 **행동(어느 팀, 어떤 우선순위)** 까지
- LSTM 게이트 셋 — forget(버릴 것), input(저장할 것), output(꺼낼 것)
- padding_idx = 0 으로 패딩 임베딩 고정, hidden[-1] 의 -1 은 **층** 인덱스
- n_layers = 1 에 dropout > 0 이면 적용될 자리가 없다
- **훈련 데이터로 평가한 정확도는 성능이 아니다**
- 한글 폰트 + unicode_minus = False 는 세트, import 셀에 고정
- seq2seq는 교사 강요 비율로 학습과 추론의 간극을 조절
- 손실 계산 시 <sos> 제외([:, 1:]), 패딩은 ignore_index = 0
- 초기 손실은 ln(어휘 크기) 근처가 정상